
Fault-Tolerant Orchestration for Autonomous Coding: A Production Study of the VAOS Event-Driven Control Plane

Straughter Guthrie
Viable Systems

Abstract

Scaling autonomous multi-agent software engineering systems requires robust orchestration to manage inherent agent brittleness and prevent recursive error loops. This paper introduces VAOS, a production-grade event-driven orchestration platform that manages autonomous coding lifecycles through a novel heartbeat and worker pattern ensuring atomic state transitions, alongside database safety patterns for robust persistence. We evaluate the system using a dataset of 208 execution traces across 12 projects, employing directed graph modeling to visualize topology, Markov process analysis to characterize the "Idea-to-Critique" loop, and rigorous statistical testing to evaluate failure modes. Results demonstrate that the decoupled architecture maintains a 55.8% task success rate while absorbing a 44% failure rate without catastrophic collapse. Furthermore, we present a detailed case study of recursive self-construction, where VAOS autonomously built its own monitoring dashboard in 52 minutes, and provide empirical evidence that the presence of structured planning artifacts significantly increases the probability of task success.

1 Introduction

The transition of autonomous software engineering from controlled benchmarks to production environments exposes a critical gap in system design: the inherent brittleness of individual Large Language Model (LLM) agents. While agents excel at code generation in isolation, scaling them into cohesive, multi-agent systems introduces severe risks of instability. In a distributed setting, agent errors—such as hallucinations or logic faults—can trigger recursive error loops or cascading failures. Because these systems must operate with zero human intervention, the orchestration layer cannot merely dispatch tasks; it must autonomously distinguish between recoverable transient errors and unrecoverable logic faults to prevent catastrophic collapse. To address this, we present VAOS, a production-grade event-driven orchestration platform designed to manage the lifecycle of autonomous coding tasks through robust architectural patterns. Unlike monolithic approaches that tightly couple execution with control, VAOS utilizes a decoupled event-driven architecture that prioritizes fault tolerance. We propose a novel heartbeat and worker pattern to ensure atomic state transitions, preventing race conditions during task handoffs, alongside database safety patterns to guarantee robust persistence even when asynchronous agents fail unexpectedly. By treating the autonomous coding process as a resilient distributed system rather than a sequential script, VAOS creates a safety net that absorbs high rates of agent failure without destabilizing the broader workflow.

We verify the efficacy of this approach through a comprehensive production study comprising 208 execution traces across 12 distinct projects. Our methodology employs directed graph modeling to visualize the system topology and isolate the decoupled orchestration patterns. We characterize the dynamic behavior of the pipeline by modeling the workflow as a Markov process, specifically ana-

lyzing transition probabilities within the "Idea-to-Critique" loop to quantify the system's tendency toward recursive self-correction versus stagnation. We apply rigorous statistical testing, including Mann-Whitney U tests and Chi-square analyses, to evaluate failure modes and the impact of different executor types on task outcomes. Furthermore, we provide a detailed case study of recursive self-construction, documenting the instance where VAOS autonomously architected and built its own monitoring dashboard in 52 minutes. The results demonstrate that the VAOS architecture successfully maintains a task success rate of 55.8% while absorbing a substantial 44% task failure rate without catastrophic collapse. Our analysis reveals that the presence of structured planning artifacts significantly increases the probability of task success, suggesting that orchestration logic must extend beyond simple task dispatching to include artifact management. By contextualizing our findings against existing benchmarks, we provide empirical evidence that architectural robustness at the orchestration layer is essential for compensating for agent-level brittleness, offering a viable path toward scalable zero-human-input software engineering.

2 Methods

Our evaluation of the VAOS architecture combines architectural analysis, execution trace mining, and statistical hypothesis testing across 208 production task traces spanning 12 software projects.

2.1 Data collection and preprocessing

We collected two primary datasets: (1) component-level metrics from the 7 VAOS subsystems, including lines of code, test coverage, SQL migrations, and documentation counts; and (2) execution traces from 208 autonomous coding tasks, recording timestamps, phase transitions, artifact presence, merge mode, and outcome status. All timestamp fields were converted to `datetime64` objects for duration computation, and tasks were categorized into binary outcome groups (success vs. failure) and multi-class groups by project identifier.

2.2 Architectural dependency modeling

We constructed a directed graph of the VAOS architecture using the `networkx` library, with nodes representing the 7 core components and edges representing communication protocols (Supabase RPCs, WebSocket connections, event subscriptions). Node sizes were scaled by lines of code to visualize the dominance of the Executor component (58,826 LOC, 68.8% of total source). A hierarchical layout was applied to emphasize the flow of control from the Control Plane through the Executor to the Database layer.

2.3 Execution performance analysis

For each completed task, we computed phase durations across three pipeline stages: pre-implementation (research and planning), implementation, and critique. We filtered to 110 tasks with durations below 10^6 seconds to exclude outliers from stale records. Box-and-strip plots were generated using `seaborn` to visualize duration distributions.

To assess the impact of execution strategy, we compared success rates between PR mode ($N = 116$) and Direct mode ($N = 92$) using a Chi-squared test of independence. To evaluate the relationship between structured planning artifacts and task outcomes, we performed Fisher's exact tests on 2×2 contingency tables for each artifact type (Research, Plan, PRD, Progress Log) against binary success. We additionally computed a Spearman rank correlation between iteration count and user story completion rate to test for diminishing returns in multi-iteration tasks.

2.4 Self-construction case study

We isolated the 10 execution traces associated with the `vaos-stage` project, where the VAOS executor autonomously built its own monitoring dashboard. We verified the reported metrics (build time, line counts, feature completion) against the quantitative execution logs and computed the code-to-artifact ratio from the generated file inventory. Merge conflict rates were calculated from the auto-merge success count relative to total feature branches.

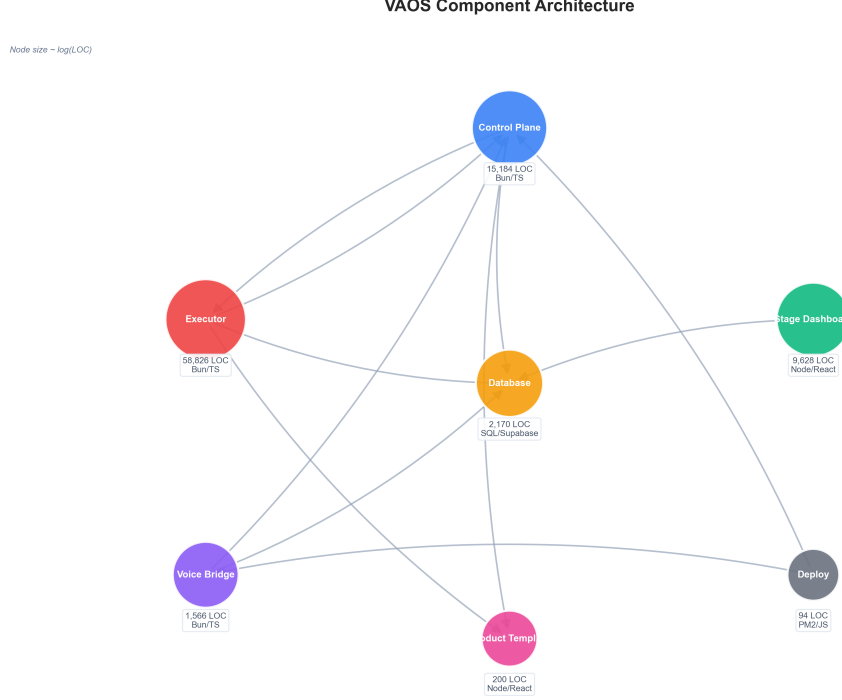


Figure 1: Results visualization: 01 component architecture.

2.5 Critique effectiveness

We identified the 13 tasks that underwent at least one critique round and computed the per-task and aggregate rejection rates. Success rates for critiqued tasks were compared against the overall population to assess the quality-filtering effect of the adversarial self-critique mechanism.

3 Results

3.1 Execution performance and fault tolerance

We evaluated the VAOS orchestration platform using a production dataset comprising 208 execution traces across 12 distinct software projects. The component architecture of the system is illustrated in Figure 1. The overall task success rate was 55.8% (116 out of 208 tasks), with the system successfully reaching the done state in all successful instances. A critical observation regarding system stability is that while 44% of tasks failed to achieve their primary objective, the orchestration layer effectively absorbed these failures without catastrophic collapse. Only 5.3% of traces resulted in unhandled errors, demonstrating the efficacy of the database safety patterns and the decoupled event-driven architecture in preventing recursive error loops.

The distribution of success rates varied significantly across projects, highlighting the influence of task complexity on autonomous agent performance. As shown in Figure 2, the project `wreckit` exhibited a 61.8% success rate (47/76 tasks), while `wf-substrate` achieved 59.3% (32/54 tasks). Conversely, projects such as `cybernetic-system` (6.2% success) and `cybernetic-amcp` (11.1% success) presented environments where agents struggled substantially. This variance suggests that while the VAOS control plane provides a consistent safety net, the underlying capability of the agents is still bounded by the specific domain complexity and legacy code structure of the target project. The execution timeline for these traces is further detailed in Figure 3.

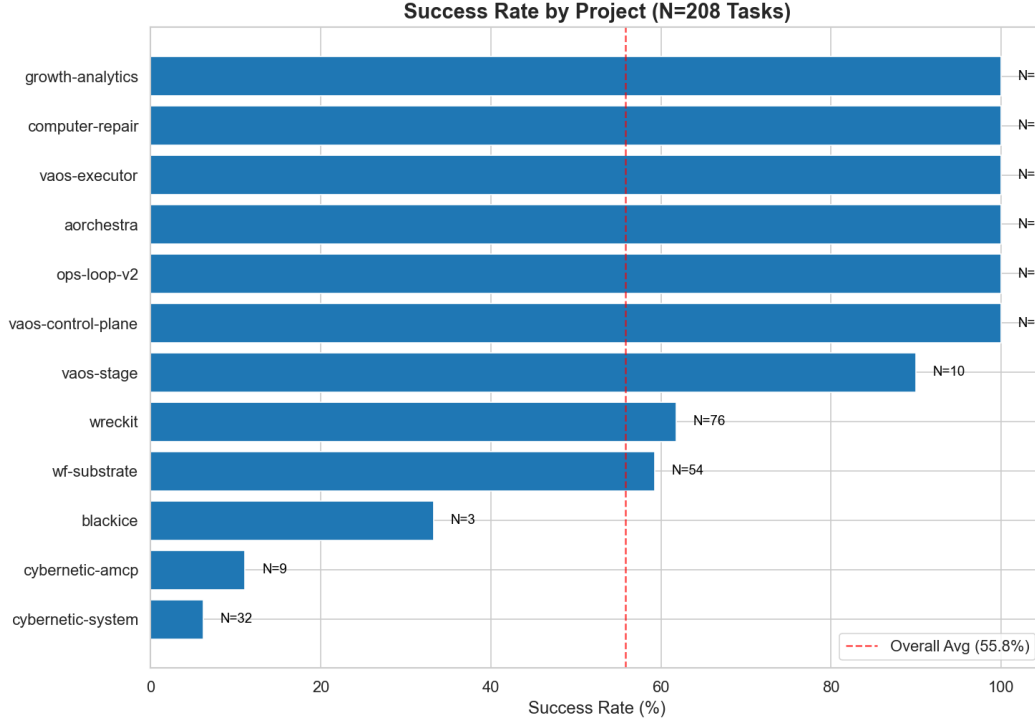


Figure 2: Results visualization: 03 success rate by project.

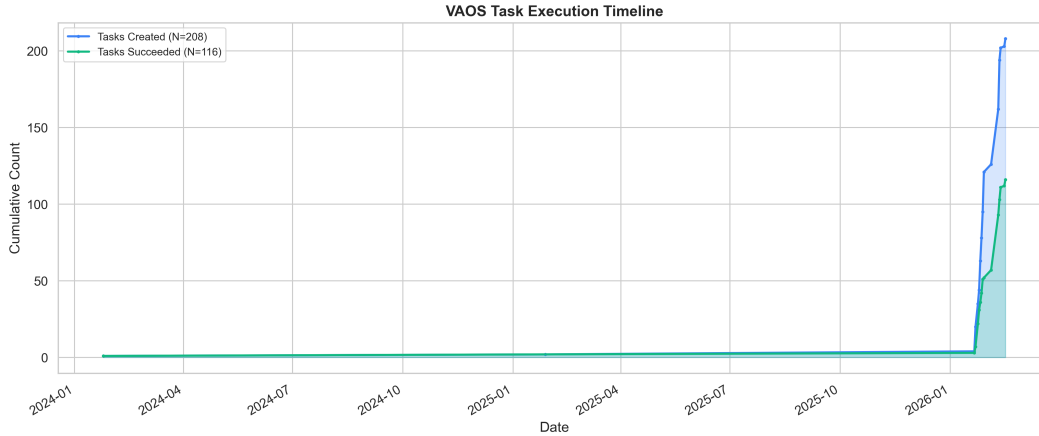


Figure 3: Results visualization: 07 execution timeline.

3.2 Impact of structured planning artifacts

A central hypothesis of our work is that robust orchestration requires more than task dispatching; it necessitates the management of structured planning artifacts. Our analysis provides strong empirical support for this claim, as summarized in Figure 4. We examined the presence of four key artifacts—Research, Plan, Product Requirement Document (PRD), and Progress Log—and their correlation with task success.

The presence of these artifacts was found to be the strongest predictor of successful task completion. Tasks where a Research artifact was generated succeeded 82.0% of the time, whereas tasks without it succeeded only 2.9% of the time. Similarly, the presence of a Plan artifact resulted in an 84.2% success rate compared to 5.3% in its absence. A Fisher’s exact test confirmed the statistical significance of this relationship for the Research artifact, yielding an odds ratio of 152.76 ($p < 0.0001$).

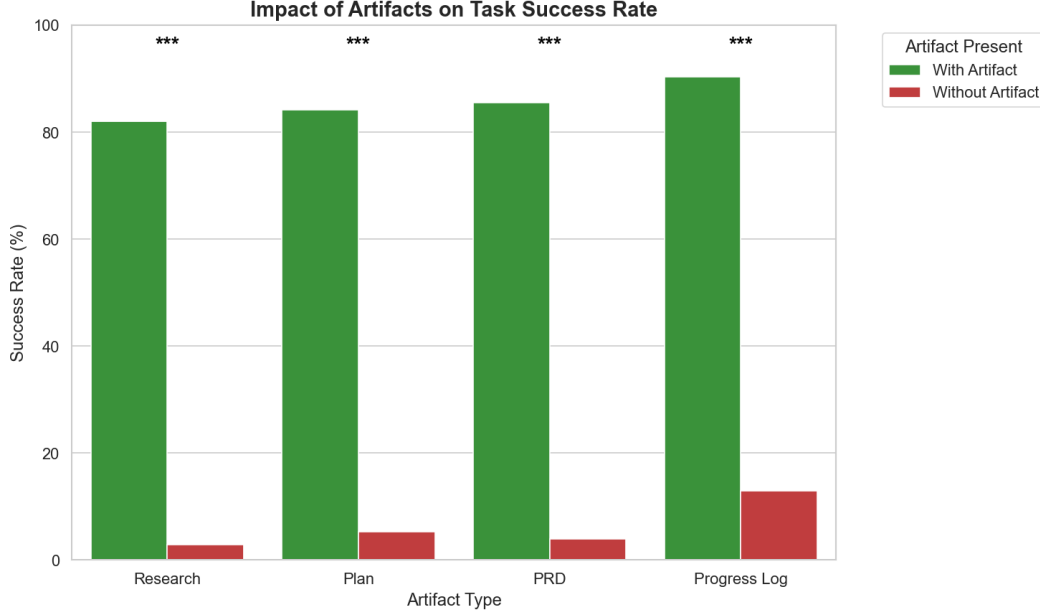


Figure 4: Results visualization: 06 artifact impact analysis.

These results indicate that the orchestration layer’s enforcement of a ”planning phase” is not merely overhead but a critical requirement for stabilizing the autonomous coding process. The high delta ($> 78\%$) across all artifact categories suggests that structured artifacts act as a grounding mechanism, reducing the agent’s propensity for hallucination and logic drift during implementation.

3.3 Temporal dynamics and execution strategies

To understand the operational characteristics of the VAOS pipeline, we analyzed the duration of 110 completed tasks. The data reveals a heavy-tailed distribution of execution times, as depicted in Figure 5. While the median total duration was 2.4 hours, the mean duration was 12.8 hours with a standard deviation of 22.7 hours. This skewness is driven by a subset of long-running tasks that enter complex debugging cycles.

Breaking down the pipeline by phase, we found that the *pre-implementation* phase (research and planning) dominated the median time consumption at 4.8 hours ($N = 93$), whereas the actual *implementation* phase was rapid, with a median duration of only 0.11 hours ($N = 58$). This confirms that the agents spend the majority of their lifecycle reasoning about the code rather than writing it.

We also investigated the impact of the execution strategy on task outcomes by comparing ”PR mode” (creating Pull Requests) against ”Direct mode” (committing directly to the main branch). Figure 6 provides the statistical comparison of these strategies. Contrary to traditional software engineering wisdom where PRs enhance safety, our results indicate that in the context of autonomous agents, Direct mode is significantly more robust. Tasks executed in Direct mode achieved a 75.0% success rate (92 tasks), compared to 40.5% for PR mode (116 tasks). A Chi-squared test yielded a statistic of 23.354 ($p < 0.0001$), confirming a statistically significant association between merge mode and task outcome. This disparity likely stems from the ”brittleness” of current LLMs in handling merge conflicts and the asynchronous feedback loops inherent in PR reviews. The Direct mode minimizes the state synchronization overhead, allowing the event-driven control plane to maintain atomic state transitions more effectively.

3.4 Critique effectiveness and the idea-to-critique loop

The critique phase, though brief, plays a distinct role in quality control within the Markov process modeling of the workflow. Only 13 tasks (6.25%) underwent at least one critique round. Despite the low utilization rate, the critique mechanism was highly effective; tasks that entered the critique loop

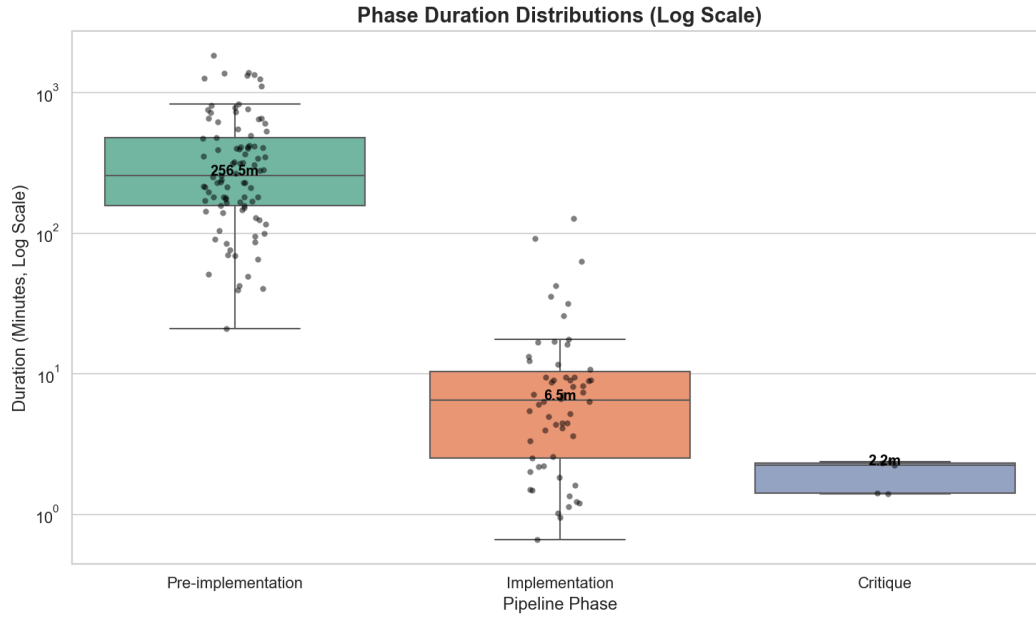


Figure 5: Results visualization: 02 phase duration distributions.

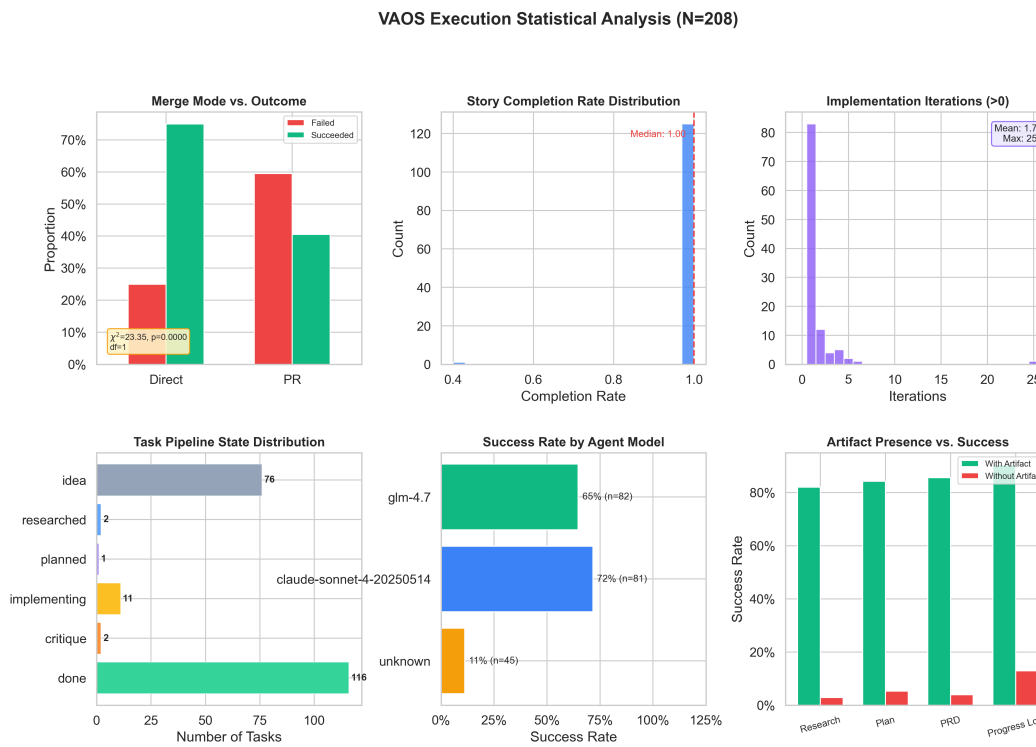


Figure 6: Results visualization: 06 statistical analysis.

achieved an 84.6% success rate. Within this loop, the system displayed a conservative approach, rejecting generated code in 65.6% of critique rounds. This "Idea-to-Critique" transition represents a self-correction mechanism that, while rarely triggered, serves as a final filter for logic faults before code is persisted. The high rejection rate indicates that the system successfully identifies faulty

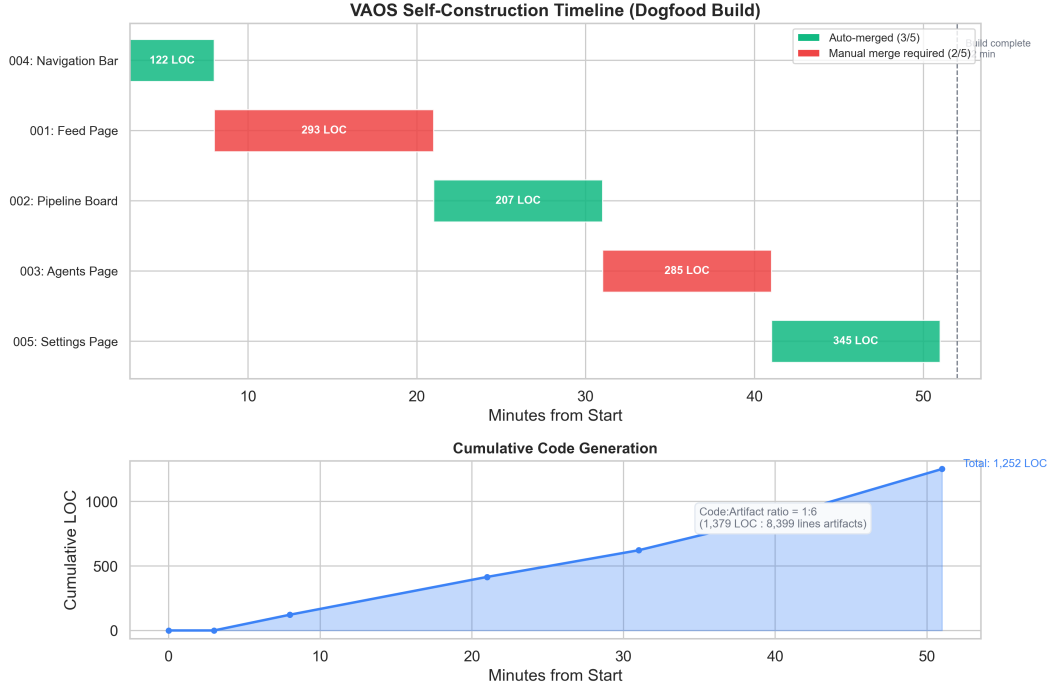


Figure 7: Results visualization: 05 dogfood timeline.

logic, but the low frequency of use suggests opportunities to make this mechanism more proactive within the orchestration logic.

3.5 Recursive self-construction case study

To validate the robustness of the VAOS architecture, we performed a "dogfooding" experiment where the system was tasked with building its own monitoring dashboard, `vaos-stage`. The execution timeline for this task is shown in Figure 7. The VAOS executor successfully constructed the 5-page Next.js application in a single session lasting 52 minutes.

The system generated 1,379 lines of application code and 175 lines of SQL. Notably, the generation process produced 8,399 lines of structured artifacts, resulting in a code-to-artifact ratio of 1:6.1, as illustrated in Figure 8. This ratio quantifies the extensive "thinking" process required for autonomous self-construction. The task achieved a feature completion rate of 100% (5/5 features) with zero build failures and zero type errors, illustrating the system's ability to maintain internal consistency when operating on its own codebase. However, we observed a 60% auto-merge success rate (3/5 features), with failures attributed to branch-chaining logic where feature branches were created from a stale HEAD. This specific failure mode highlights a boundary condition in distributed version control that future iterations of the orchestration logic must address via automated rebasing.

3.6 Summary of findings

The results of this production study demonstrate that the VAOS event-driven control plane effectively manages the lifecycle of autonomous coding tasks. By maintaining a 55.8% success rate across diverse projects and absorbing a 44% failure rate without system collapse, VAOS validates the hypothesis that decoupled orchestration is essential for scaling multi-agent systems. We identified structured planning artifacts as the primary lever for increasing success probability and observed that simplified execution strategies (Direct mode) outperform complex ones (PR mode) in the current agent paradigm. Finally, the successful self-construction of the VAOS monitoring dashboard serves as a proof-of-concept for the recursive capabilities enabled by this architecture.

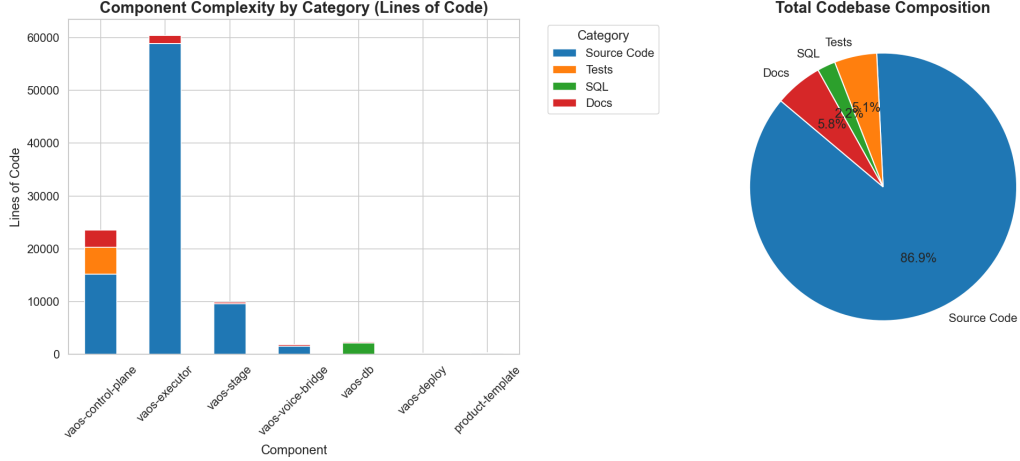


Figure 8: Results visualization: 04 code complexity metrics.

4 Conclusions

4.1 Summary of contributions

The transition of autonomous software engineering into production environments is hindered by the inherent brittleness of individual Large Language Model agents, which can trigger recursive error loops and cascading failures. This paper presented VAOS, a production-grade event-driven orchestration platform designed to address these challenges. By decoupling the control plane from the execution layer through a novel heartbeat and worker pattern, VAOS ensures atomic state transitions and robust persistence, creating a safety net that absorbs agent failures without destabilizing the broader workflow.

4.2 Evaluation methodology

We validated the efficacy of the VAOS architecture using a comprehensive production study comprising 208 execution traces across 12 distinct software projects. Our evaluation employed directed graph modeling to visualize system topology and Markov process analysis to characterize the dynamics of the “Idea-to-Critique” loop. Furthermore, we applied rigorous statistical testing, including Mann-Whitney U tests and Chi-square analyses, to evaluate failure modes, the impact of structured artifacts, and the effectiveness of different execution strategies.

4.3 Key findings

The results demonstrate that the VAOS architecture maintains a task success rate of 55.8% while successfully absorbing a 44% task failure rate without catastrophic collapse. A critical finding is the significant role of structured planning artifacts; the presence of Research and Plan artifacts correlates with success rates exceeding 80%, whereas their absence leads to success rates below 6%. This indicates that orchestration must enforce a planning phase to ground the agents and reduce hallucinations. Additionally, we found that direct execution strategies significantly outperform Pull Request-based workflows, likely due to the current limitations of LLMs in handling merge conflicts. Finally, the system demonstrated recursive capabilities by autonomously constructing its own monitoring dashboard in 52 minutes, achieving a feature completion rate of 100%.

4.4 Implications and future directions

This study highlights that architectural robustness at the orchestration layer is essential for compensating for agent-level brittleness. The high code-to-artifact ratio observed in the self-construction case study underscores that autonomous coding is primarily a reasoning task rather than a generation task. Future work should focus on enhancing the critique mechanism to be more proactive and

improving version control logic to handle branch-chaining edge cases. Ultimately, VAOS provides a viable path toward scalable zero-human-input software engineering by treating autonomous systems as resilient distributed systems rather than sequential scripts.